Guía Práctica: Primer App mobile en React Native con Expo

Configuración del entorno de desarrollo, instalación y ejecución de nuestro primer "hola Mundo" en un dispositivo móvil.

En esta clase daremos nuestros primeros pasos con React Native utilizando Expo. La intención es que al finalizar el ejercicio, tener una primera aplicación corriendo directamente en un teléfono móvil.

1. Requisitos Previos en la Computadora

Antes de iniciar, verificamos que nuestro equipo cuente con el software básico instalado:

- • **Node.js (Versión LTS):** Descargable e instalable desde nodejs.org (seleccionar la opción LTS).
  1. Ve al sitio web oficial: nodejs.org.
  2. Descarga el instalador **LTS** (Long Term Support) para Windows (.msi).
  3. Ejecuta el archivo descargado y sigue el asistente:
     1. Acepta los términos de la licencia.
     2. Mantén la ruta de instalación por defecto (C:\Program Files\nodejs\).
     3. **Paso crítico:** En la pantalla *Custom Setup*, asegúrate de que la opción **"Add to PATH"** esté seleccionada con el ícono de disco.
     4. Finaliza el instalador.
  4. Para verificar que Node.js esté correctamente instalado en el sistema, abre PowerShell o CMD y ejecuta:

```
node -v
npm -v
```

Deberías ver la versión de Node (ej. `v20.x.x` o superior) y la versión de `npm`.

- • **Visual Studio Code:** Entorno de desarrollo recomendado, disponible en code.visualstudio.com.

  Si no lo tienes instalado, descarga **Visual Studio Code** desde code.visualstudio.com.
  - Ejecuta el instalador y, en la sección de tareas adicionales, se recomienda marcar:
  - **"Agregar 'Abrir con Code' al menú contextual de archivos/directorios"**.
  - **"Agregar a PATH"**.

- • **Extensiones en VS Code:** Instalar desde la pestaña de extensiones la extensión 'Expo Tools' (brinda autocompletado y soporte en archivos de configuración).

  Abre VS Code e instala las siguientes extensiones recomendadas desde el menú de *Extensions* (`Ctrl + Shift + X`):
  - **Expo Tools** (ayuda con autocompletado y archivos de configuración de Expo).
  - **ES7+ React/Redux/React-Native snippets** (atajos de código útil).

2. Preparación del Teléfono Móvil

- Abre la tienda de aplicaciones de tu celular (Google Play Store en Android o App Store en iOS).
- Busca e instala la aplicación gratuita Expo Go.
- **Requisito de Red Fundamental:** Asegúrate de que el celular esté conectado a la misma red Wi-Fi que la computadora de desarrollo.

3. Creación del Proyecto (Versión Congelada)

Abrir la terminal (PowerShell o CMD) en la carpeta donde guardamos los proyectos ejecutamos el comando exacto de inicialización:

```
npx create-expo-app@5.0.0 mi-primer-app --template blank@57
```

Tambien podría haberse usado:

```
npx create-expo-app@latest mi-primer-app --template blank
```

En ese caso se hubiera creado la app con la version “latest”, pero como vimos en el caso de las imágenes de contenedores Docker, es una buena práctica espacificar el “tag” de versión, así de esa forma nos aseguramos que todo el equipo de desarrollo no tendrá problemas de compatibilidad. Lo mismo con la versión de SDK que es el valor “57” que figuea luego del caracter @

> 📌 **Nota de compatibilidad sobre el SDK 57**
>
> El comando fija tanto la versión de la herramienta generadora (2.0.1) como la plantilla del SDK 57 de Expo. La etiqueta @57 coincide exactamente con el SDK soportado por la aplicación Expo Go que acaban de instalar en sus teléfonos desde la tienda, garantizando compatibilidad directa sin inconsistencias de versiones.

---

4. Ejecución del Servidor de Desarrollo



Veamos que hay en el directorio `mi-primer-app/`

📁 ***`assets/` (Recursos visuales)***

- **¿Qué es?** Es la carpeta donde se guardan todos los archivos estáticos que va a usar la aplicación.
- **¿Qué hay adentro?** Imágenes (`.png`, `.jpg`), tipografías personalizadas (`.ttf`, `.otf`), audios, videos o íconos. Por defecto, Expo ya te deja acá el ícono de la app (`icon.png`) y la pantalla de carga (`splash-screen.png`).

📁 ***`node_modules/` (Librerías y dependencias externas)***

- **¿Qué es?** Es el "almacén" gigante donde se descargan todas las herramientas y librerías que tu proyecto necesita para funcionar.
- **¿Qué hay adentro?** Muchas subcarpetas con el código de **React**, **React Native**, el propio **Expo** y cualquier otra librería que instalemos en el futuro. *Nota : ¡Esta carpeta nunca se edita a mano y no se sube a GitHub porque es pesadísima! Se regenera sola con `npm install`. y debe figurar en .gitignore*

📄 ***`.gitignore` (Filtro para Git/GitHub)***

- **¿Qué es?** Un archivo de texto que le dice a Git qué carpetas y archivos debe ignorar y **no** subir al repositorio en línea.
- **¿Qué hay adentro?** Por defecto, tiene anotada la carpeta `node_modules/`, archivos de configuración del sistema (`.DS_Store` en Mac, `thumbs.db` en Windows) y carpetas temporales de Expo (`.expo/`).

`App.js` (El corazón de tu app)

- **¿Qué es?** Es el punto de entrada principal del código de nuestra aplicación.
- **¿Qué hay adentro?** El primer componente de React Native que se renderiza en la pantalla. Cuando abrimos el proyecto en VS Code, este es el archivo donde vamos a escribir nuestro "Hola Mundo", diseñar nuestras pantallas y programar la lógica inicial.

`app.json` (La cédula de identidad de la app)

- **¿Qué es?** El archivo de configuración global de Expo.
- **¿Qué hay adentro?** Información del proyecto como el nombre visible de la app, la versión, el ícono, el color de fondo de la pantalla de carga, y configuraciones específicas para cuando la quieras compilar y publicar en Google Play (Android) o App Store (iOS).

`index.js` (El interruptor de encendido)

- **¿Qué es?** Es el archivo que arranca todo tras bambalinas.
- **¿Qué hay adentro?** Registra el componente principal (`App.js`) dentro del entorno de React Native para que el celular o el emulador sepan por dónde empezar a ejecutar el código. Por lo general, no hace falta tocarlo.

`LICENSE` (Términos de uso)

- **¿Qué es?** Un documento de texto que especifica bajo qué condiciones y permisos otras personas pueden usar, modificar o distribuir el código de tu proyecto (por ejemplo, una licencia MIT si es software libre).

`package-lock.json` (El registro de seguridad)

- **¿Qué es?** Un historial exacto e inmutable de cada mini-librería instalada dentro de `node_modules/`.
- **¿Qué hay adentro?** Registra las versiones exactas y los enlaces de descarga de absolutamente todo lo que instaló `npm`. Sirve para garantizar que si otro compañero de equipo (o el profesor) descarga el proyecto, se le instale exactamente lo mismo y no rompa el código por diferencias de versiones.

`package.json` (La receta del proyecto)

- **¿Qué es?** El manifiesto principal y el cerebro administrativo del proyecto.
- **¿Qué hay adentro?** Tiene tres cosas fundamentales:
    1. **Datos básicos:** El nombre de la app y la versión.
    2. **Scripts:** Atajos para la terminal (como `npm run start` o `npm run android`).
    3. **Dependencies (Dependencias):** La lista de "ingredientes" (librerías) que tu app necesita para funcionar (como `expo`, `react`, `react-native`).

- Ingresa a la carpeta del proyecto recién creado:

```
cd mi-primer-app
```

Debería crearse una estructura de directorios así:

```
mi-primer-app/
├── assets/
├── node-modules/
├── .gitignore
├── App.js          <-- Punto de entrada de la aplicación
├── app.json
├── index.js
├── LICENSE
├── package-lock.json
└── package.json
```

- Abre la carpeta en Visual Studio Code:

```
code .
```

- Inicia el servidor de desarrollo (Metro Bundler) ejecutando en la terminal integrada de VS Code:

```
npx expo start
```

En la consola verás un código QR dibujado en formato ASCII junto a las opciones de interacción.

5. Visualización en el Celular

- Abre la app Expo Go en tu celular.
- **En Android:** Android: Toca el botón 'Scan QR code' y apunta la cámara al código QR de la consola de tu computadora.
- **En iOS:** iOS: Abre la app de Cámara nativa del iPhone, apunta al QR y toca la notificación emergente para abrir en Expo Go.
- Espera unos segundos mientras se descarga el JavaScript Bundle inicial (Building JavaScript bundle...).

6. Código del "Hola Mundo"

Abre el archivo App.js en la raíz de tu proyecto, reemplaza su contenido por el siguiente código y guarda los cambios (Ctrl + S) para ver la actualización instantánea (Fast Refresh) en tu celular:

```
import React from 'react';
import { StyleSheet, Text, View, StatusBar } from 'react-native';
```

```
export default function App() {
  return (
    <View style={styles.container}>
      <StatusBar barStyle="dark-content" />
      <Text style={styles.titulo}>¡Hola Mundo!</Text>
      <Text style={styles.subtitulo}>Mi primera app en React Native + Expo</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f5f5f5',
    alignItems: 'center',
    justifyContent: 'center',
    padding: 20,
  },
  titulo: {
    fontSize: 32,
    fontWeight: 'bold',
    color: '#2c3e50',
    marginBottom: 10,
  },
  subtitulo: {
    fontSize: 16,
    color: '#7f8c8d',
    textAlign: 'center',
  },
});
```

Deberíamos ver en la pantalla de nuestro móbil la siguiente App



Parte 2: Formulario e Interacción

Ahora daremos un paso más allá del 'Hola Mundo' estático. Construiremos una aplicación interactiva que recibe el nombre del usuario mediante un campo de texto y cambia dinámicamente de pantalla para mostrar un saludo personalizado.

1. Objetivos de Aprendizaje

• **Estado local (useState):** Aprender a guardar y actualizar datos ingresados por el usuario.
• **Entrada de datos (TextInput):** Utilizar el componente TextInput y capturar cambios con onChangeText.
• **Botones personalizados (TouchableOpacity):** Crear botones con estilos personalizados usando TouchableOpacity.
• **Renderizado condicional:** Alternar vistas sin necesidad de librerías de navegación complejas.
• **Teclado interactivo (KeyboardAvoidingView):** Evitar que el teclado en pantalla tape los elementos del formulario.

2. Instrucciones para la Práctica

- Creamos un nuevo proyecto 'appFormulario' con:

```
npx create-expo-app@5.0.0 appFormulario –template blank@57
```

- En el directorio appFormulario abrimos el archivo App.js.

- Reemplaza todo el contenido existente de App.js por el código que se presenta a continuación y guarda los cambios (Ctrl + S)
- Corremos el servidor de expo con:

```
npx expo start
```

3. Código Completo (App.js)

```
import React, { useState } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  TouchableOpacity,
  StatusBar,
  KeyboardAvoidingView,
  Platform
} from 'react-native';

export default function App() {
  // Estado para guardar el nombre tipeado
  const [nombre, setNombre] = useState('');

  // Estado para controlar si ya se envió el formulario y mostrar el saludo
  const [saludado, setSaludado] = useState(false);

  // Función al presionar el botón de Ingresar
  const handleIngresar = () => {
    if (nombre.trim() !== '') {
      setSaludado(true);
    }
  };

  // Función para volver a la pantalla inicial
  const handleVolver = () => {
    setNombre('');
    setSaludado(false);
  };

  return (
```

```
    <KeyboardAvoidingView
      behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
      style={styles.container}
    >
      <StatusBar barStyle="dark-content" />

      {!saludado ? (
        /* --- PANTALLA 1: FORMULARIO --- */
        <View style={styles.card}>
          <Text style={styles.titulo}>Bienvenido/a</Text>
          <Text style={styles.subtitulo}>Por favor, ingresá tu nombre para
continuar:</Text>

          <TextInput
            style={styles.input}
            placeholder="Ej: Osvaldo"
            placeholderTextColor="#a0aec0"
            value={nombre}
            onChangeText={setNombre}
            autoCapitalize="words"
          />

          <TouchableOpacity
            style={[styles.boton, nombre.trim() === '' && styles.botonDeshabilitado]}
            onPress={handleIngresar}
            disabled={nombre.trim() === ''}
          >
            <Text style={styles.textoBoton}>Ingresar</Text>
          </TouchableOpacity>
        </View>
      ) : (
        /* --- PANTALLA 2: SALUDO --- */
        <View style={styles.card}>
          <Text style={styles.saludoEmoji}>👋</Text>
          <Text style={styles.tituloSaludo}>¡Hola, {nombre}!</Text>
          <Text style={styles.subtitulo}>¡Qué bueno verte por acá!</Text>

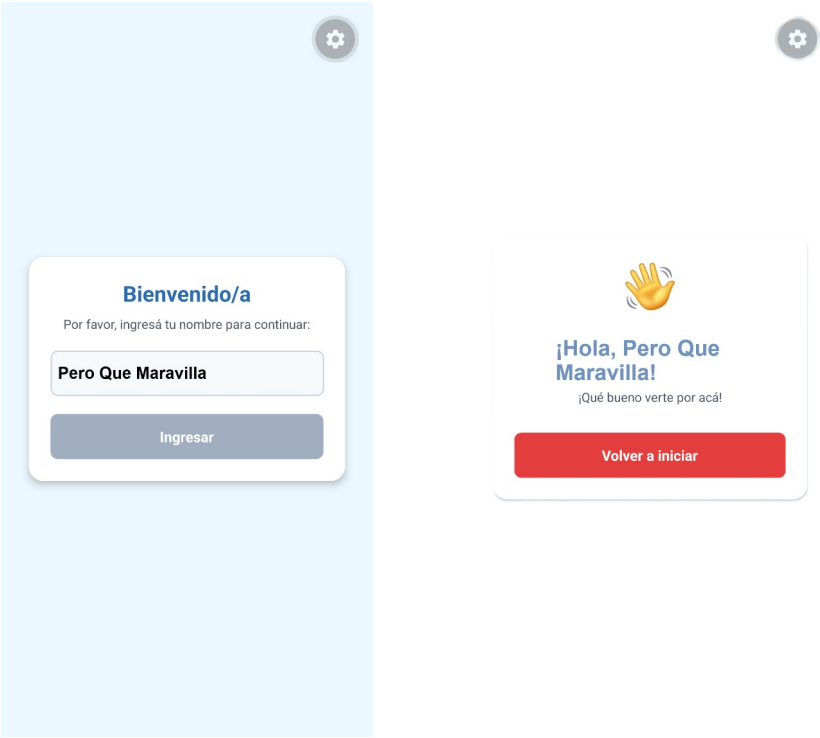
          <TouchableOpacity
            style={[styles.boton, styles.botonVolver]}
            onPress={handleVolver}
          >
            <Text style={styles.textoBoton}>Volver a iniciar</Text>
```

```
          </TouchableOpacity>
        </View>
      )}
    </KeyboardAvoidingView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#ebf8ff', // Azul claro de fondo
    alignItems: 'center',
    justifyContent: 'center',
    padding: 20,
  },
  card: {
    backgroundColor: '#ffffff',
    borderRadius: 16,
    padding: 24,
    width: '100%',
    maxWidth: 350,
    alignItems: 'center',
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.1,
    shadowRadius: 10,
    elevation: 5, // Sombra para Android
  },
  titulo: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#2b6cb0',
    marginBottom: 8,
  },
  saludoEmoji: {
    fontSize: 48,
    marginBottom: 10,
  },
  tituloSaludo: {
    fontSize: 26,
    fontWeight: 'bold',
    color: '#2c5282',
```

```
    marginBottom: 8,
    textAlign: 'center',
  },
  subtitulo: {
    fontSize: 14,
    color: '#4a5568',
    textAlign: 'center',
    marginBottom: 20,
  },
  input: {
    width: '100%',
    height: 50,
    borderColor: '#cbd5e0',
    borderWidth: 1.5,
    borderRadius: 8,
    paddingHorizontal: 15,
    fontSize: 16,
    color: '#2d3748',
    backgroundColor: '#f7fafc',
    marginBottom: 20,
  },
  boton: {
    width: '100%',
    height: 50,
    backgroundColor: '#3182ce',
    borderRadius: 8,
    alignItems: 'center',
    justifyContent: 'center',
  },
  botonDeshabilitado: {
    backgroundColor: '#a0aec0',
  },
  botonVolver: {
    backgroundColor: '#e53e3e', // Rojo para la acción de volver
    marginTop: 10,
  },
  textoBoton: {
    color: '#ffffff',
    fontSize: 16,
    fontWeight: 'bold',
  },
});
```

Visualmente quedarán así:



de esa forma hemos dado el primer paso en el desarrollo de código en el “mundo mobile”.

4. Explicación de Conceptos Clave

> 💡 **TextInput vs HTML Input**
>
> En React Native, el evento onChangeText pasa directamente el valor de texto tipeado (string), a diferencia de la web donde se recibe un evento sintético (e.target.value).

> 💡 **TouchableOpacity vs Button**
>
> El componente Button nativo es rígido en cuanto a diseño. TouchableOpacity permite envolver cualquier diseño y otorga una retroalimentación visual reduciendo su opacidad al ser presionado.

> 💡 **KeyboardAvoidingView**
>
> Este componente ajusta automáticamente la posición de la pantalla cuando se despliega el teclado del dispositivo móvil, evitando que el TextInput quede oculto.

5. Solución a Problemas Frecuentes en el Aula

⚠ Alerta de Firewall de Windows

Al ejecutar 'npx expo start' por primera vez, Windows mostrará una ventana emergente de seguridad. Es necesario permitir el acceso marcando tanto redes privadas como públicas.

⚠ Restricciones en la Red Wi-Fi / Aislamiento de Clientes

Si el celular no logra conectar al servidor de la PC (error de timeout), se puede iniciar Expo en modo túnel ejecutando: 'npx expo start --tunnel'. Esto canaliza la conexión mediante servidores externos evitando bloqueos de red local.